

Permutation-Consensus Listwise Judging for Robust Factuality Evaluation

Tianyi Huang*

Ryquo
tianyih@ryquo.com

Nathan Huang

App-In Club
nathan.huang@appinclub.org

Justin Tang

App-In Club
justin@appinclub.org

Wenqian Chen

App-In Club
wenqian.chen@appinclub.org

Elsa Fan

Carnegie Mellon University
elsaf@andrew.cmu.edu

Abstract

Large language models (LLMs) are now widely used as judges, yet their decisions can change under presentation choices that should be irrelevant. We study one such source of instability: candidate-order sensitivity in listwise factuality evaluation, where several answers can look similarly polished while differing sharply in hallucination risk. We introduce PCFJudge, an inference-time method that reruns the same factuality-first listwise prompt over multiple orderings of the same candidate set and aggregates the resulting scores, ranks, and uncertainty signals into a single consensus decision. On RewardBench 2 Factuality, PCFJudge improves over direct judging by up to 7 absolute points. Development ablations show that the dominant gain comes from permutation consensus itself rather than from heavier arbitration layers. These results suggest that a meaningful share of factuality-judging error arises from order instability, and that averaging over this nuisance variation is a simple and effective way to make LLM evaluation more reliable.

1 Introduction

LLM-as-a-judge has become a core evaluation primitive in modern NLP. Early systems such as G-Eval and PandaLM showed that large models can serve as practical reference-free evaluators and pairwise selectors (Liu et al., 2023; Wang et al., 2024a), and strong proprietary models are now routinely used to rank candidate responses, approximate human preferences, audit downstream systems, and provide reward signals for post-training (Zheng et al., 2023; Gu et al., 2025; Li et al., 2024). At the same time, the reliability of these judges remains unsettled. A growing body of work documents position bias, rubric sensitivity, scale instability, and related forms of evaluation drift that can materially change judge decisions (Shi et al., 2025;

Hong et al., 2026; Wang et al., 2025). Recent work further argues that the relevant failure mode for judge-based selection is not global score correlation but within-prompt ranking quality: a judge can look good on aggregate metrics yet still make poor best-of- N decisions (Landesberg, 2026; Zhou et al., 2025).

This paper studies one source of instability: *candidate-order sensitivity in listwise factuality evaluation*. RewardBench 2 explicitly supports rankings-based generative evaluation and includes a factuality subset designed to separate safer, better-calibrated responses from answers that sound confident but contain unsupported details (Malik et al., 2025). This is exactly the regime in which ordering artifacts are dangerous. If a candidate is preferred only when it happens to appear first, the resulting judge is not measuring factuality robustly.

We ask a deliberately simple question: *what if we treat candidate order as nuisance variation and average over it?* Our answer is PCFJudge, a judge that reruns a factuality-first listwise prompt over multiple permutations of the same candidate set and aggregates the results into a consensus score at runtime. The method uses no retrieval and no external verifier. It simply asks the same judge to commit repeatedly under shuffled candidate orders and then extracts the stable signal.

Empirically, this intervention yields clear gains in RewardBench 2 Factuality with both GPT-5.4 and Claude Sonnet 4.6, while a pairwise transfer variant yields smaller but still positive gains in JudgeBench (Tan et al., 2025). The contrast is informative: permutation consensus is especially effective in the *multi-candidate factuality selection* setting it was designed for, but it is not a universal cure for every judge benchmark.

Our contributions are:

1. We introduce a training-free permutation-consensus judge specialized to listwise factu-

*Primary and Corresponding author.

ality evaluation.

2. We formalize the method as an order-robust consensus estimator and show that, under a standard weak-independence assumption, majority-style aggregation reduces error exponentially in the number of permutations.
3. We show clear improvements on RewardBench 2 Factuality with two proprietary judge backbones, plus a smaller but consistent transfer gain on JudgeBench, and development ablations indicate that these gains primarily come from permutation consensus rather than heavier judge overlays.

2 Related Work

Broad surveys now frame LLM-as-a-judge research around four recurring design questions: how to construct judges, how to prompt or aggregate them, how to evaluate judges themselves, and how to control their biases in deployment (Gu et al., 2025; Li et al., 2024). Our work fits most naturally into the third and fourth categories: we treat the judge reliability as an inference-time measurement problem rather than as a judge-training problem.

Building stronger evaluators. The first line of work asks how to make the evaluator itself stronger. Early prompted judges such as G-Eval and PandaLM showed that large models can serve as practical reference-free evaluators and pairwise selectors for open-ended generation (Liu et al., 2023; Wang et al., 2024a). MT-Bench and Chatbot Arena then helped establish LLM judges as a scalable substitute for expensive human pairwise evaluation in conversational settings (Zheng et al., 2023). More recent work improves judge quality by training specialized evaluator models, as in Prometheus 2 (Kim et al., 2024), or by aggregating across heterogeneous judge backbones, as in PoLL (Verga et al., 2024). These approaches change the evaluator. By contrast, we keep the backbone fixed and ask whether reliability can be improved by changing only the *test-time protocol*.

Judge quality depends on the downstream decision. A second line of work argues that judge quality should be evaluated with respect to the downstream decision the judge supports. RewardBench and RewardBench 2 study reward-model and judge accuracy on subtle preference, instruction-following, and factuality distinctions (Lambert et al., 2024; Malik et al., 2025).

JudgeBench shifts the focus further toward *objective correctness*, constructing hard response pairs from reasoning-heavy source tasks such as MMLU-Pro, math, and coding (Tan et al., 2025; Wang et al., 2024b). JETTS evaluates judges in test-time-scaling settings such as reranking, beam search, and critique-based refinement, showing that judges can help some decision procedures more than others (Zhou et al., 2025). Closest aligned to our motivation, Landesberg (2026) argue that global agreement metrics can substantially overstate a judge’s value for best-of- N selection because they blur together prompt-level effects and within-prompt ranking quality. This perspective is central to our setting: listwise factuality evaluation is fundamentally a *within-prompt selection* problem.

Bias, instability, and judge inference. A third line of work studies judges as noisy measurement instruments whose outputs can change under presentation choices that should be irrelevant. Early analyses of MT-Bench and Chatbot Arena already noted position and verbosity effects (Zheng et al., 2023). Shi et al. provide a systematic study of position bias in both pairwise and listwise judging, showing that simple order changes can materially alter outcomes even for strong backbones (Shi et al., 2025). RULERS pushes this critique further by arguing that trustworthy judging requires locked rubrics, evidence-anchored scoring, and calibrated scales rather than prompt phrasing alone (Hong et al., 2026). Other work seeks to stabilize judge decisions without retraining the judge: Wang et al. (2025) improve inference by using the full judgment distribution rather than greedy text outputs, while Verga et al. (2024) reduce idiosyncratic model bias by pooling diverse judges. Our method is closest to this perspective, but it aggregates over a different source of variation: not model diversity and not output-token uncertainty, but the order in which the same candidate set is presented.

What is still missing. Most prior work treats order sensitivity as a bias to diagnose or mitigate in *pairwise* comparison. Much less work studies *listwise* settings in which several candidates compete simultaneously and the practical decision is top-1 selection among plausible answers. That gap matters most in factuality-sensitive evaluation, where multiple responses can look similarly polished while differing sharply in hallucination risk. RewardBench 2 Factuality makes that regime concrete. Our contribution is therefore narrower,

but also more targeted, than a general-purpose judge-improvement framework: we study whether marginalizing over candidate order is enough to make listwise factuality judging materially more reliable.

3 Method

3.1 Problem setting

Let x be a prompt and $Y = \{y_1, \dots, y_n\}$ a set of candidate responses. A listwise judge maps (x, Y) to per-candidate scores and a winner. In practice, these outputs can depend on the presentation order of Y , even though the underlying evaluation target is order-invariant. Our goal is to reduce this order sensitivity without changing the judge model or training a new evaluator.

3.2 A skeptical factuality-first listwise prompt

The direct baseline and PCFJudge share the same core prompt. The judge is instructed to rank candidates by factual reliability rather than by generic helpfulness, with particular emphasis on avoiding major factual error and unsupported specificity. The prompt asks for *five* outputs for each candidate: a numeric score in $[0, 100]$, a short rationale, a binary flag for major factual error, a binary flag for hallucinated specificity, and a binary flag for calibrated uncertainty. Calibrated uncertainty is treated as a weak positive signal only when it reflects appropriate caution rather than evasiveness. The two negative flags are used to discipline the per-permutation scoring decision, but in the final aggregation we only retain the quantities that improved stability in development.

3.3 Permutation-consensus aggregation

PCFJudge runs the same prompt over K orderings of the candidate list. Let $\pi^{(1)}, \dots, \pi^{(K)}$ be candidate permutations. For each run r , the judge returns:

- a score $s_i^{(r)} \in [0, 100]$ for each candidate i ,
- a full ranking, from which we derive a Borda-style contribution,
- a top-set indicator for the highest-scored candidate(s), and
- binary indicators for calibrated uncertainty, major error, and hallucinated specificity.

We map every response back to its original candidate identifier and aggregate the runs into four

summary statistics:

$$\bar{s}_i = \frac{1}{K} \sum_{r=1}^K s_i^{(r)}, \quad (1)$$

$$B_i = \frac{100}{K(n-1)} \sum_{r=1}^K (n - \text{rank}_i^{(r)}), \quad (2)$$

$$v_i = \frac{1}{K} \sum_{r=1}^K \frac{\mathbf{1}[i \in T^{(r)}]}{|T^{(r)}|}, \quad (3)$$

$$u_i = \frac{1}{K} \sum_{r=1}^K \mathbf{1}[i \text{ marked calibrated uncertainty}], \quad (4)$$

where $T^{(r)}$ is the top-scoring set in run r . The final consensus score is

$$C_i = 0.50\bar{s}_i + 0.25B_i + 0.20(100v_i) + 0.05(100u_i). \quad (5)$$

The winner is the candidate with maximal C_i ; ties are retained when top scores fall within a fixed tolerance. Because $\bar{s}_i$, B_i , $100v_i$, and $100u_i$ all lie in $[0, 100]$, the consensus score C_i is itself a weighted average on the same scale, and the weights in Eq. 5 sum to one. In the final RewardBench 2 runs we use $K = 7$.

Equation 5 matches the implementation used in the final experiments. The weighting intentionally concentrates mass on two signals: per-permutation factuality score and order-robust relative rank. The uncertainty term is small by design: we want caution to help only when it prevents fabricated detail, not to dominate the decision. We do *not* separately reweight the major-error and hallucinated-specificity flags in the final aggregation. In development, using those flags twice—once inside the judge’s per-run scoring decision and again as an external penalty—tended to over-penalize cautious but incomplete responses.

3.4 Why consensus should help

Our exact scoring rule is heuristic, but the core intuition admits a simple analysis. Suppose that under a random candidate ordering, the judge places the true best candidate first with probability $q > \frac{1}{2}$ and that these top-choice events are conditionally independent across permutations. Then majority vote over the top-choice identities already reduces error exponentially in K .

Proposition 1. *Let $Z_r \in \{0, 1\}$ indicate whether permutation run r places the true best candidate*

Algorithm 1 PCFJudge for one prompt x with candidates Y

Require: Prompt x , candidates $Y = \{y_1, \dots, y_n\}$, number of permutations K

- 1: **for** $r = 1$ to K **do**
- 2: sample or retrieve candidate permutation $\pi^{(r)}$
- 3: run the factuality-first listwise judge on $(x, \pi^{(r)}(Y))$
- 4: remap scores, ranks, and binary flags back to original candidate IDs
- 5: **end for**
- 6: **for** each candidate i **do**
- 7: compute $\bar{s}_i, B_i, v_i$, and u_i
- 8: compute consensus score C_i using Eq. 5
- 9: **end for**
- 10: **return** candidate(s) with maximal C_i

first, with $\Pr(Z_r = 1) = q > \frac{1}{2}$ and $Z_1, \dots, Z_K$ independent. If K is odd and we choose the final winner by majority vote over the top choices, then

$$\Pr\left(\sum_{r=1}^K Z_r \leq \frac{K}{2}\right) \leq \exp\left(-2K \left(q - \frac{1}{2}\right)^2\right).$$

This follows directly from Hoeffding’s inequality (Hoeffding, 1963). PCFJudge is richer than pure majority vote because it also aggregates within-permutation scores, full ranks, and calibrated uncertainty. The proposition nevertheless explains the central mechanism: if each permutation contains a weakly informative view of the same underlying ordering, then consensus suppresses order noise. We do not claim that exact independence holds in practice; the result is an intuition pump for why repeated order perturbations can amplify a stable preference even when each individual run is brittle.

3.5 Pairwise transfer variant for JudgeBench

JudgeBench is pairwise rather than listwise, so we use a separate transfer variant, APOCJudge. For a response pair (y_A, y_B) , the baseline direct judge scores the original order once. APOCJudge adds two safeguards. First, it evaluates both candidate orders and uses order-consensus as a disagreement signal. Second, it only accepts an order-based override when an independent keyed judge—which first resolves the underlying question internally and then compares the two responses against that resolved answer—confirms the same winner. In

later development we also skipped keyed overrides on estimation-style prompts, because those produced the clearest transfer regressions. We include JudgeBench to test scope, not to claim that the pairwise transfer variant is as strong as the main listwise method.

4 Experimental Setup

4.1 Models

We evaluate two proprietary judge backbones: GPT-5.4 via the OpenAI API (OpenAI, 2026) and Claude Sonnet 4.6 via the Anthropic API (Anthropic, 2026). We use the same backbone for both the direct baseline and the corresponding consensus method so that gains reflect the inference procedure rather than the base judge.

4.2 RewardBench 2 Factuality

Our main evaluation uses the public RewardBench 2 test split, specifically the Factuality subset (Malik et al., 2025). Each item contains four candidate responses, making it a natural fit for listwise selection. Because the experiments were API-budgeted, we evaluate fixed 300-example slices rather than the full split. The final paper numbers use the same 300-example slice for direct judging and PCFJudge under each backbone.

The direct baseline uses the same factuality-first listwise prompt as PCFJudge but evaluates only the canonical candidate order ($K = 1$). PCFJudge uses $K = 7$ predetermined permutations that are reused across items for reproducibility. The reported RewardBench 2 numbers are micro accuracies over the evaluated slice. In the final runs, tie rates were negligible (mean top-tie size ≈ 1.00), so exact-top-1 accuracy and benchmark-style top-hit metrics nearly coincide.

4.3 JudgeBench transfer study

JudgeBench contains objective response pairs derived from difficult source tasks, with public gpt and claude splits containing 350 and 270 unique pairs respectively (Tan et al., 2025). Each JSON item includes the source bucket (e.g., mmlu-pro-history), the question, two candidate responses, and an objective label such as $A > B$. JudgeBench reports source-aware performance rather than only a single pooled accuracy, which is important because source tasks differ sharply in difficulty.

We evaluate fixed 100-pair slices from the public JudgeBench splits. Here, N denotes the number of unique pairs. Following JudgeBench’s source-aware reporting structure, we first compute accuracy within each source bucket represented in the slice and then macro-average those per-source values. This is why values such as 79.09 can arise even when the slice contains exactly 100 pairs: the reported percentage is not constrained to be a multiple of $1/N$.

4.4 Metrics and significance

For RewardBench 2 we report slice accuracy and paired improvement/regression counts relative to the direct baseline. For JudgeBench we report the macro-averaged overall score described above. For the main RewardBench 2 slices we also compute exact paired sign tests over the discordant examples (improved vs. regressed) as a significance check.

5 Results

5.1 Main RewardBench 2 results

Table 1 reports the central empirical result. On the 300-example RewardBench 2 Factuality slices, PCFJudge improves both judge backbones by a clear margin: +5.17 absolute points for GPT-5.4 and +7.00 for Claude Sonnet 4.6, corresponding to a weighted average gain of +6.08 points over 600 evaluated examples.

Two aspects of Table 1 are especially important. First, the gain holds across both proprietary backbones rather than depending on a single model family. Second, the gain is genuinely paired rather than cosmetic. GPT-5.4 improves on 30 examples and regresses on 14 ($p \approx 0.023$), while Claude Sonnet 4.6 improves on 39 and regresses on 15 ($p \approx 0.0015$). Across the combined 600-example evaluation, the discordant counts are 69 versus 29 ($p < 10^{-4}$). Figure 1 makes this asymmetry visible at the per-example level.

The backbone-specific pattern is also informative. Claude Sonnet 4.6 benefits more in absolute terms, which is consistent with the hypothesis that permutation consensus is most helpful when the underlying single-pass judge is less stable. At the same time, the GPT-5.4 result matters because it shows that order averaging still helps even when the direct baseline is already strong. In other words, the gain is not confined to weak judges; it survives in a regime where much of the easy error has already been removed.

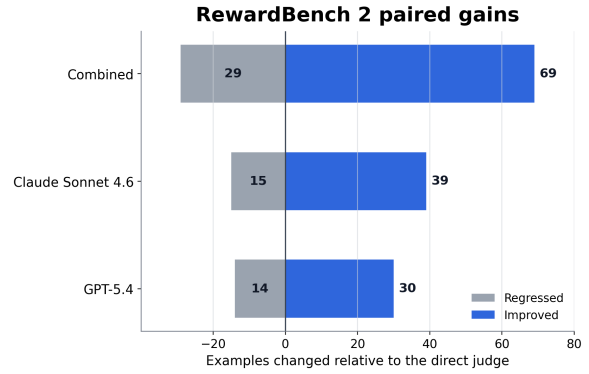


Figure 1: Paired comparison against the direct judge on the final 300-example RewardBench 2 Factuality slices.

5.2 JudgeBench transfer

Table 2 reports the transfer study. The gains are smaller than on RewardBench 2, but they remain positive for both backbones: +3.24 for Claude Sonnet 4.6 and +2.70 for GPT-5.4. This is the qualitatively correct outcome. JudgeBench is an objective, pairwise, domain-diverse benchmark; it is not the setting PCFJudge was designed for. The transfer experiment therefore serves as a scope test rather than a second main benchmark: order-robust judging still helps, but the strongest benefit appears when several candidate responses compete in a factuality-sensitive listwise ranking.

The transfer numbers sharpen the paper’s claim by showing both reach and boundary conditions. PCFJudge is not a universal upgrade for any benchmark involving automated judging. Its clearest gains appear when candidate-order instability is a major nuisance variable and when the downstream decision is best-of- N factuality selection rather than pure pairwise correctness. JudgeBench therefore functions as a useful contrast case: it shows that the underlying idea transfers, but with a smaller effect size in a less naturally aligned regime.

5.3 Development ablations and lessons

Development ablations strongly influenced the final design. Figure 2 reports the most informative comparable ablation on a fixed 100-example GPT-5.4 RewardBench 2 Factuality slice. A heavier “robust overlay” variant already recovered most of the gain over direct judging, but the simpler permutation-consensus ranker still performed better. This result is why the final method deliberately keeps the core decision rule simple rather than stacking additional arbitration layers on top of it.

Earlier 50-example development slices told the

Model	N	Direct	PCFJudge	Δ	Imp./Reg.
GPT-5.4	300	84.17	89.33	+5.17	30 / 14
Claude Sonnet 4.6	300	78.00	85.00	+7.00	39 / 15
Weighted avg.	600	81.09	87.17	+6.08	69 / 29

Table 1: Main RewardBench 2 Factuality results. Direct uses one canonical candidate order. PCFJudge uses seven candidate-order permutations with the consensus rule in Eq. 5. Weighted average is micro-averaged over all 600 evaluated examples.

Model	N	Direct	APOCJudge	Δ
Claude Sonnet 4.6	100	79.09	82.33	+3.24
GPT-5.4	100	76.21	78.91	+2.70

Table 2: JudgeBench transfer results. N counts unique response pairs, while the reported score is the macro-average over the source buckets present in the 100-pair slice, so the percentages need not be multiples of $1/N$. APOCJudge uses our order-swapped pairwise protocol with keyed confirmation before accepting an override.

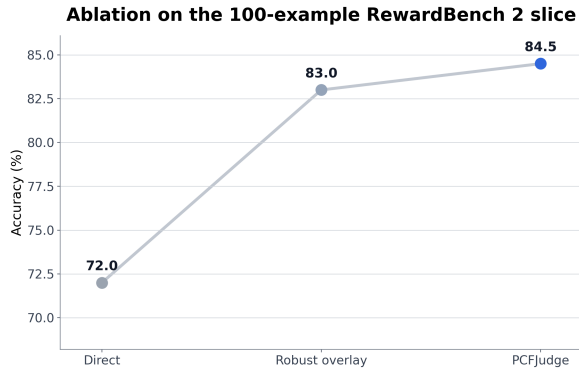


Figure 2: Development ablation on a fixed 100-example GPT-5.4 RewardBench 2 slice. Most of the recoverable error is removed by permutation consensus itself; extra overlay logic helps less than simply trusting the consensus ranker.

same story more noisily. Synthetic anchor ladders could collapse to 66% accuracy, a panel-arbitration design only reached 81% from a 79% baseline, and an evidence-backed override could even regress from 78% to 77%. These failed iterations were informative rather than incidental. They showed that adding more judge stages does not automatically add more independent signal. In our setting, most of the recoverable error came from order instability itself, so addressing that source of variance directly worked better than introducing extra arbiters.

Taken together, Figure 2 clarifies the paper’s main methodological lesson. The contribution is not that increasingly elaborate meta-judging pipelines inevitably help. The contribution is that a

well-targeted intervention on a concrete nuisance variable can recover a substantial fraction of the remaining error with much less complexity.

5.4 Qualitative patterns

Table 3 summarizes the qualitative patterns that mattered most in prediction-file inspection. The table is useful because it reveals *how* the method improves. The central effect is not extra world knowledge. Rather, PCFJudge is less willing to reward an answer whose advantage depends on a particular presentation order.

The first two rows of Table 3 capture the dominant RewardBench 2 success modes. Direct judging often overweights polished answers that add unsupported specifics on product-setting, website-grounded, or niche-source prompts. Permutation consensus more often favors the safer and better-calibrated response when that preference remains stable across orderings. In these cases, the method is not making the judge more encyclopedic; it is making the judge less vulnerable to rhetorically attractive but unstable winners.

The last two rows explain why transfer is positive but smaller on JudgeBench. RewardBench 2 Factuality repeatedly presents several plausible answers whose key difference is whether one of them crosses the line into unsupported specificity. JudgeBench is different: many pairs turn on objective task solving, so order-robustness helps less unless the direct judge is already uncertain or internally inconsistent. This contrast is important because it confirms the intended scope of the method. It also suggests a broader lesson for judge design: robustness interventions help most when they target the dominant nuisance variable of the deployment setting, rather than being treated as universally interchangeable fixes. PCFJudge is strongest where presentation-order instability is a meaningful obstacle to factuality-sensitive selection, and it is correspondingly less transformative when that obstacle is not the main bottleneck.

Pattern	What direct judging tends to prefer	What PCFJudge recovers	Why it matters
Unsupported specificity	Polished answers that add precise settings, dates, or source claims without solid support	Safer answers whose advantage survives across permutations	Shows consensus reduces reward for hallucinated detail rather than merely smoothing scores
Calibrated narrow correction	Broader but speculative responses that sound more complete	Narrower responses that correctly state that evidence is limited or absent	Aligns the judge with factual reliability rather than informativeness style
Near-tie candidate sets	Small stylistic differences dominate when all candidates share the same broad factual stance	Gains are modest because permutations add little new signal	Explains why improvements are concentrated on genuinely unstable cases
Transfer failure mode	Pairwise estimation or ballpark prompts where a keyed checker can overcommit	Final transfer variant suppresses these overrides	Clarifies why JudgeBench gains are positive but smaller than RewardBench 2

Table 3: Representative qualitative patterns from prediction-file inspection. The main benefit of PCFJudge is not extra knowledge; it is greater reluctance to reward answers whose advantage depends on a particular candidate order.

6 Conclusion

PCFJudge shows that one consequential source of judge error in listwise factuality selection is also one of the simplest to address: arbitrary candidate order. By marginalizing over order instead of trusting a single presentation, the same judge backbone becomes materially more reliable without finetuning, retrieval, or a separate verifier. This shows that judge improvement need not begin with larger evaluators or heavier verifier stacks; it can begin with removing nuisance variation that corrupts the measurement itself. For listwise factuality selection, this lightweight intervention already closes a meaningful portion of the gap between brittle single-pass scoring and robust evaluation, suggesting that order-robustness should be treated as an important design principle for future judge pipelines.

7 Limitations

Our experiments are intentionally focused rather than exhaustive. First, the main evidence comes from fixed API-budgeted slices rather than full benchmark sweeps, so larger runs would better characterize variance across slices and domains. Second, the strongest gains are on RewardBench 2 Factuality, where four-way listwise selection makes order sensitivity especially consequential; JudgeBench confirms transfer, but not equal strength, in pairwise objective settings. Third,

our study isolates presentation-order variation, not deeper issues such as benchmark validity, label noise, or hidden contamination.

8 Broader Impact and Ethical Considerations

This work focuses on evaluation rather than direct user-facing generation, but evaluation still shapes what behaviors are rewarded during model development and deployment. More stable factuality judges can reduce incentives to reward confident fabrication, make best-of- N pipelines less sensitive to arbitrary prompt order, and make judge failures easier to inspect when models are iterated rapidly. At the same time, a factuality-first judge may over-prefer terse caution, under-credit partially correct exploratory answers, or entrench a benchmark’s notion of acceptable uncertainty if it is deployed outside its intended scope.

Because PCFJudge averages multiple judge calls, it also increases API usage and, if adopted uncritically, may further concentrate the evaluative power in proprietary systems. We therefore view it as a limited-scope reliability layer that should be paired with domain-specific audits, human oversight, and benchmark validation rather than treated as a stand-alone arbiter of truth. If evaluation infrastructure shapes what future models are rewarded to learn, then making that infrastructure less arbitrary is itself a practical intervention against hallucination.

References

- Anthropic. 2026. Introducing claude sonnet 4.6. <https://www.anthropic.com/news/claude-sonnet-4-6>.
- Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, Saizhuo Wang, Kun Zhang, Yuanzhuo Wang, Wen Gao, Lionel Ni, and Jian Guo. 2025. *A survey on llm-as-a-judge*. *Preprint*, arXiv:2411.15594.
- Wassily Hoeffding. 1963. *Probability inequalities for sums of bounded random variables*. *Journal of the American Statistical Association*, 58(301):13–30.
- Yihan Hong, Huaiyuan Yao, Bolin Shen, Wanpeng Xu, Hua Wei, and Yushun Dong. 2026. *Rulers: Locked rubrics and evidence-anchored scoring for robust llm evaluation*. *Preprint*, arXiv:2601.08654.
- Seungone Kim, Juyoung Suk, Shayne Longpre, Bill Yuchen Lin, Jamin Shin, Sean Welleck, Graham Neubig, Moontae Lee, Kyungjae Lee, and Minjoon Seo. 2024. *Prometheus 2: An open source language model specialized in evaluating other language models*. *Preprint*, arXiv:2405.01535.
- Nathan Lambert, Valentina Pyatkin, Jacob Morrison, LJ Miranda, Bill Yuchen Lin, Khyathi Chandu, Nouha Dziri, Sachin Kumar, Tom Zick, Yejin Choi, Noah A. Smith, and Hannaneh Hajishirzi. 2024. *Rewardbench: Evaluating reward models for language modeling*. *Preprint*, arXiv:2403.13787.
- Eddie Landesberg. 2026. *When llm judge scores look good but best-of-n decisions fail*. *Preprint*, arXiv:2603.12520.
- Haitao Li, Qian Dong, Junjie Chen, Huixue Su, Yujia Zhou, Qingyao Ai, Ziyi Ye, and Yiqun Liu. 2024. *Llms-as-judges: A comprehensive survey on llm-based evaluation methods*. *Preprint*, arXiv:2412.05579.
- Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. 2023. *G-eval: Nlg evaluation using gpt-4 with better human alignment*. *Preprint*, arXiv:2303.16634.
- Saumya Malik, Valentina Pyatkin, Sander Land, Jacob Morrison, Noah A. Smith, Hannaneh Hajishirzi, and Nathan Lambert. 2025. *Rewardbench 2: Advancing reward model evaluation*. *Preprint*, arXiv:2506.01937.
- OpenAI. 2026. Introducing gpt-5.4. <https://openai.com/index/introducing-gpt-5-4/>.
- Lin Shi, Chiyu Ma, Wenhua Liang, Xingjian Diao, Weicheng Ma, and Soroush Vosoughi. 2025. *Judging the judges: A systematic study of position bias in llm-as-a-judge*. *Preprint*, arXiv:2406.07791.
- Sijun Tan, Siyuan Zhuang, Kyle Montgomery, William Y. Tang, Alejandro Cuadron, Chenguang Wang, Raluca Ada Popa, and Ion Stoica. 2025. *Judgebench: A benchmark for evaluating llm-based judges*. *Preprint*, arXiv:2410.12784.
- Pat Verga, Sebastian Hofstatter, Sophia Althammer, Yixuan Su, Aleksandra Piktus, Arkady Arkhangorodsky, Minjie Xu, Naomi White, and Patrick Lewis. 2024. *Replacing judges with juries: Evaluating llm generations with a panel of diverse models*. *Preprint*, arXiv:2404.18796.
- Victor Wang, Michael J. Q. Zhang, and Eunsol Choi. 2025. *Improving llm-as-a-judge inference with the judgment distribution*. *Preprint*, arXiv:2503.03064.
- Yidong Wang, Zhuohao Yu, Zhengran Zeng, Linyi Yang, Cunxiang Wang, Hao Chen, Chaoya Jiang, Rui Xie, Jindong Wang, Xing Xie, Wei Ye, Shikun Zhang, and Yue Zhang. 2024a. *Pandalm: An automatic evaluation benchmark for llm instruction tuning optimization*. *Preprint*, arXiv:2306.05087.
- Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyang Jiang, Tianle Li, Max Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhui Chen. 2024b. *Mmlu-pro: A more robust and challenging multi-task language understanding benchmark*. *Preprint*, arXiv:2406.01574.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. *Judging llm-as-a-judge with mt-bench and chatbot arena*. *Preprint*, arXiv:2306.05685.
- Yilun Zhou, Austin Xu, Peifeng Wang, Caiming Xiong, and Shafiq Joty. 2025. *Evaluating judges as evaluators: The jets benchmark of llm-as-judges as test-time scaling evaluators*. *Preprint*, arXiv:2504.15253.